
Endogenous Signals Fail to Allocate: Explicit Difficulty Supervision for Token-Level Adaptive Compute

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Token-level adaptive compute promises to make the transformer forward pass
2 resource-aware, but most mechanisms key off *endogenous* signals: attention en-
3 tropy, router scores, confidence estimates. Their correlation with problem diffi-
4 culty is assumed, not tested. We test the assumption on arithmetic with graded
5 difficulty, a clean testbed in which difficulty grades deterministically from 1-digit
6 addition to 4×3 -digit multiplication, ground-truth labels exist for every token,
7 and pretraining knowledge is absent. The endogenous signals fail in three distinct
8 ways. An entropy-driven gate *collapses* to a constant (per-token entropy–difficulty
9 correlation $r \approx -0.06$); a tag-initialized gate *inverts*, spending more compute on
10 easy problems than hard ones ($r = -0.59$) because the cross-entropy gradient
11 is largest on memorizable tokens; and a learned difficulty head trained without
12 supervision is *vacuous*, with allocation unstable across seeds. Explicit per-token
13 difficulty supervision under a budget constraint fixes all three: gates order cor-
14 rectly (easy < medium < hard) in every seed with per-problem gate–difficulty
15 $r = 0.95$, accuracy matches the full-compute baseline while cutting FLOPs per
16 token by 42%, and it beats a static schedule at the same compute budget. When
17 difficulty labels or proxies exist, supervised allocation is simple, reliable, and in-
18 terpretable.

19 1 Introduction

20 Resource-aware agentic AI is mostly practiced *outside* the model: cascade queries across models
21 of different sizes [Chen et al., 2023], truncate chains of thought, cap tool calls, budget retries. The
22 forward pass itself remains resource-blind. A transformer [Vaswani et al., 2017] spends identical
23 FLOPs on a trivial token and on the token that decides whether the answer is right, yet difficulty is
24 unevenly distributed across tokens: carrying a digit in multi-digit multiplication is hard; copying a
25 “+” sign is not. An agent that must meter its compute should be able to allocate it per token.

26 Adaptive-compute methods show that large savings are available: early exiting [Teerapittayanon
27 et al., 2017, Xin et al., 2020, Schuster et al., 2022], adaptive computation time [Graves, 2016],
28 dynamic layer skipping [Wang et al., 2018, Elbayad et al., 2020], slimmable networks [Yu and
29 Huang, 2019], and token-level Mixture-of-Depths (MoD) routing [Raposo et al., 2024] all reduce
30 average compute per token. Nearly all of them share one assumption: a signal the network already
31 produces internally, be it attention entropy, router scores, or exit confidence, correlates with how
32 difficult a token is, so that end-to-end training on the task loss will shape that signal into a sensible
33 allocation. The assumption is rarely tested directly. We test it, and it fails.

This paper is an empirical account of a four-iteration journey toward token-level adaptive compute on arithmetic reasoning with graded difficulty (Table 1). Our testbed is a per-token continuous gate $g \in [0, 1]$ that scales each token’s feed-forward (FFN) output at every layer, with FLOPs charged proportionally to the gate: a token gated at 0.2 costs a fifth of the FFN compute of a token gated at 1.0. The question is what should drive the gate. Iteration 1 drives it with attention entropy, a label-free endogenous difficulty proxy: the gate collapses to a constant, and entropy does not encode task difficulty ($r \approx -0.06$ per token). Iteration 2 initializes the gating state from explicit difficulty tags. The gate now varies, but *inverts*: easy tokens receive more compute than hard ones ($r = -0.59$ per problem), because the cross-entropy gradient is largest on easy, memorizable tokens and end-to-end training follows the gradient, not the semantics. Iteration 3 replaces the entropy signal with a learned difficulty head while still training it only on the task loss: the gate is nearly flat, and its correlation with difficulty is unstable across seeds. Iteration 4 supervises the head explicitly with per-token difficulty targets plus a budget constraint, and it works.

Contributions.

- **A systematic diagnosis of endogenous difficulty signals.** Across three distinct instantiations (entropy-driven state, tag-initialized state, learned unsupervised head), endogenous training signals fail in three different ways: *collapse*, *inversion*, and *vacuity* (§4.2). A standard MoD top- k router exhibits the same inversion, so the pathology is a general property of endogenous training dynamics, not of our architecture.
- **A simple fix: explicit difficulty supervision with a budget constraint** (§3). A small per-layer difficulty head is trained with an MSE loss toward tag-derived per-token targets while a budget term pins the mean gate; a two-stage protocol with a nearly frozen body preserves task accuracy.
- **A controlled study** on tagged arithmetic (§4.3): gates order correctly (easy < medium < hard) in all 3 seeds with per-problem gate-difficulty $r = 0.95$; accuracy matches the full-compute baseline at 42% fewer FLOPs per token, and exceeds a static gate schedule at the same compute budget.
- **An analysis of generalization** (§4.4, §4.5): on untagged mixed-difficulty inputs a coarse digit-count proxy partially transfers (correct ordering, $r = 0.70$, small spread), and we identify when a static schedule suffices.

2 Related Work

Adaptive compute. Adaptive Computation Time [Graves, 2016] lets recurrent networks ponder per position, with a halting unit trained end-to-end. Early-exit methods attach classifiers to intermediate layers and stop when confident [Teerapittayanon et al., 2017, Xin et al., 2020, Elbayad et al., 2020, Schuster et al., 2022]. SkipNet [Wang et al., 2018] learns per-example layer skipping, and universally slimmable networks [Yu and Huang, 2019] run a single network at several widths. Mixture-of-Depths [Raposo et al., 2024] routes tokens through or around transformer blocks with a learned top- k router, allocating a fixed compute budget per sequence. All of these decide allocation from endogenous quantities, halting scores, exit confidence, router logits, trained on the task loss (sometimes with auxiliary balancing objectives). Our work is complementary: rather than proposing a new routing mechanism, we ask whether such endogenous signals actually track problem difficulty, and show that in our setting they do not unless supervised. MoD’s own router exhibits the inversion we diagnose (§4.5).

Conditional computation and mixture-of-experts. Sparsely-gated MoE [Shazeer et al., 2017] routes tokens to expert FFNs via a learned gating network, primarily to scale capacity at constant compute rather than to allocate by difficulty; load-balancing losses prevent collapse to a few experts, an early sign that end-to-end router dynamics do not self-organize into the intended allocation. Our DifficultyHead is easy to mistake for this kind of routing, so it is worth separating the two: MoE dispatches each token to a subset of expert parameters, changing *which* capacity processes it, whereas our head changes *how much* compute the token receives and leaves the parameters in place. Attaching a difficulty predictor to expert selection is a natural combination; we leave it to future work.

86 **Entropy and confidence as uncertainty proxies.** Attention entropy varies systematically across
 87 tokens and heads [Clark et al., 2019, Vig and Belinkov, 2019], and predictive confidence is widely
 88 used as a difficulty or calibration proxy [Guo et al., 2017], e.g. for early exits. We test attention
 89 entropy directly as a *task-difficulty* signal and find it nearly uncorrelated with ground-truth difficulty
 90 ($r \approx -0.06$ per token; §4.2).

91 **Resource-aware agents.** Orchestration-level resource awareness, whether model cascades and
 92 routing [Chen et al., 2023] or token and tool budgets, treats each model call as fixed-cost. Token-
 93 level gating makes the call itself cost-sensitive. Our results bear on when such metering is feasible:
 94 when difficulty labels or proxies exist (curriculum tags, verifier scores, metadata), supervised allo-
 95 cation is cheap and reliable; when only endogenous signals are available, allocation may be absent,
 96 inverted, or unstable.

97 3 Method

98 3.1 Architecture and per-token gating

99 Our backbone is an 11M-parameter decoder-only transformer: $d_{\text{model}} = 384$, 6 layers, 6 attention
 100 heads, $d_{\text{ff}} = 1536$, vocabulary 22, sequence length 256. Each layer computes

$$x_2 = \text{LN}(x_1 + \text{Attn}(x_1)), \quad x_3 = \text{LN}(x_2 + g_t \cdot \text{FFN}(x_2)), \quad (1)$$

101 where $g_t \in [0, 1]$ is a per-token, per-layer gate. Compute is charged proportionally: a position gated
 102 at g is billed g times the per-token FFN cost, while attention cost is always paid in full. A gate below
 103 1 thus saves FLOPs proportionally (about 42% of total per-token FLOPs at mean gate ≈ 0.5 , since
 104 the FFN dominates the layer budget).

105 In our final method, the gate is produced by a small per-layer *DifficultyHead*: a two-layer MLP
 106 (Linear(384→96), ReLU, Linear(96→1)) applied to the post-attention hidden state x_2 , whose
 107 scalar output s_t is squashed by a sigmoid, $g_t = \sigma(s_t)$. Earlier iterations used other drivers for
 108 the gate (attention entropy propagated through a low-dimensional recurrent “chemical state”, or a
 109 tag-initialized state); §4.2 describes them and their failure modes. All variants share Eq. (1) and the
 110 FLOPs accounting, so comparisons isolate the gating signal.

111 3.2 Why endogenous signals fail: three diagnostics

112 End-to-end training offers the gate only one channel of feedback: the task loss. We identify three
 113 ways this goes wrong, developed empirically in §4.2. *Collapse*: if the driving signal carries no
 114 information about difficulty, the cheapest way to satisfy a sparsity penalty is a uniform gate, and
 115 attention entropy is such a signal ($r \approx -0.06$ per token). *Inversion*: a ground-truth initialization
 116 makes the gate vary, but easy problems are memorizable, so their cross-entropy gradient is largest
 117 and training allocates them *more* compute. *Vacuity*: a head trained only through the task loss receives
 118 no signal about what difficulty is, so the gate stays flat and unstable. §3.3 supplies both the missing
 119 signal and the incentive.

120 3.3 Explicit difficulty supervision with a budget constraint

121 Our method trains the DifficultyHeads with explicit per-token targets derived from difficulty meta-
 122 data. Training is two-stage.

123 **Stage 1 (task training).** The transformer is trained conventionally for 5000 steps with the gate fixed
 124 at 1 everywhere, and checkpointed. This decouples representation learning from allocation learning.

125 **Stage 2 (allocation training).** Starting from the checkpoint, we attach the per-layer DifficultyHeads
 126 and train for 3000 steps with

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{mse}} \frac{1}{N} \sum_t (\sigma(s_t) - \tau_t)^2 + \lambda_{\text{budget}} (\bar{g} - 0.5)^2, \quad (2)$$

127 where τ_t is a per-position target gate and $\bar{g}$ the mean gate over the batch. On tagged data, targets
 128 come from the difficulty tag: easy $\rightarrow 0.2$, medium $\rightarrow 0.5$, hard $\rightarrow 0.8$, and non-problem positions

Table 1: The four-iteration journey on tagged arithmetic: mean gate per difficulty bin (E/M/H) for each gating strategy. Only explicit difficulty supervision with a budget constraint produces the correct ordering.

Gating strategy	E	M	H	Spread
1: entropy-driven	0.232	0.233	0.233	≈ 0
2: tag-initialized	0.266	0.265	0.213	-0.05
3: learned, unsupervised	0.812	0.824	0.834	$+0.02$
4: difficulty-supervised	0.37	0.51	0.58	0.20 ± 0.03

→ 0.5. We use $\lambda_{\text{mse}} = 5.0$ and $\lambda_{\text{budget}} = 0.5$; the budget term pins the mean gate near 0.5 (hence the total compute bill) while leaving the head free to redistribute compute *across* tokens. There is no sparsity penalty: the goal is allocation, not maximal savings. Stage 2 uses a two-group AdamW schedule: the body trains at a near-frozen learning rate 3×10^{-6} and the DifficultyHeads at 3×10^{-4} ; gradient clipping is 1.0 throughout. The low body learning rate preserves Stage-1 accuracy while the heads learn allocation; the cost is that the body can no longer adapt much in Stage 2, a confound we return to in §4.4 and §5.

4 Experiments

4.1 Setup

Tasks. We study arithmetic with graded difficulty in two regimes. In the *tagged* regime each problem is prefixed with a difficulty tag: [E] 1-digit addition/subtraction (~ 162 unique problems, memorizable), [M] 3-digit addition/subtraction ($\sim 1.6\text{M}$ unique), and [H] 4-digit $\times$ 3-digit multiplication ($\sim 9\text{M}$ unique). In the *mixed* regime there are no tags: operands have 1–4 digits and operators are $\{+, -, \times\}$; the difficulty bin is the digit count of the larger operand (1–4). Sequences pack multiple problems up to 256 tokens.

Evaluation. We use a frozen held-out set of 4000 problems per difficulty bin (seed 12345, identical across methods and seeds), report accuracy on answer tokens only, and run 3 seeds for every method. We report raw and balanced (macro over bins) accuracy, MFLOPs per token under the accounting of §3.1, the mean gate per difficulty bin, and the gate spread (hardest minus easiest bin).

Baselines. All baselines train end-to-end for 6000 steps: *Baseline* (standard transformer), *Chemical-off* (gating pathway frozen, gate = 1; an architecture-matched full-compute control), *Fixed-schedule* (gate fixed at 0.5 for every token; a static schedule at the same budget as ours), *MoD* (top- k token routing [Raposo et al., 2024] at matched capacity), *Chemical-entropy* (iteration 1: entropy-driven chemical-state gate with sparsity penalty), *Random-gate* (entropy replaced by uniform noise in the same range), and *Tag-only* (gate driven only by the difficulty-tag identity, a static oracle heuristic). Our method (*Difficulty-supervised*) uses the two-stage protocol of §3.3 (5000+3000 steps on tagged). All models share the 11M backbone, batch size 32, AdamW, and gradient clipping 1.0.

Calibration. Baselines train 6000 steps end-to-end while ours trains 5000+3000 with a nearly frozen body in Stage 2; comparisons below are “same task, similar total budget”, not a controlled iso-FLOPs training study.

4.2 Diagnosing the failure modes

Table 1 summarizes the journey; Figure 1 quantifies the allocation quality per iteration.

Iteration 1: attention entropy collapses. A per-token low-dimensional chemical state, updated from position-normalized attention entropy and mapped to the gate, trained end-to-end with cross-entropy plus a sparsity penalty, converges to a constant gate: mean gates for E/M/H are 0.232/0.233/0.233 (spread ≈ 0.001). Accuracy is fine, $71.5 \pm 1.3\%$ at 11.5 MFLOPs/token, so the model saves compute but does not allocate differentially. The reason is the signal, not the mechanism: the per-token Pearson correlation between attention entropy and difficulty is ≈ -0.06 (seed range $+0.05$ to -0.12). Attention entropy does not encode task difficulty in this setting.

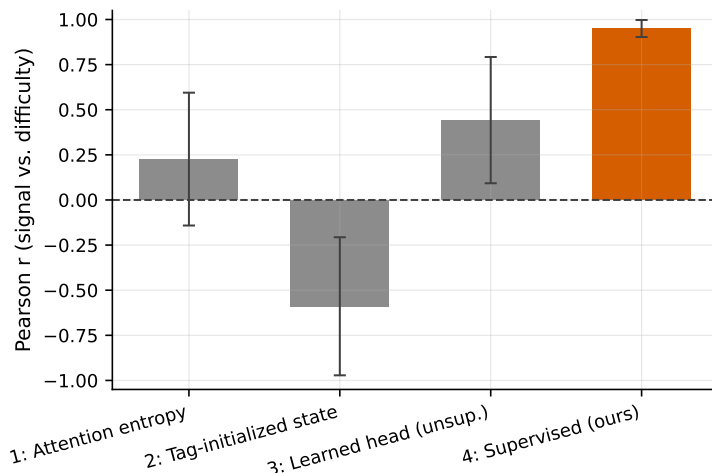


Figure 1: Per-problem correlation between mean gate and difficulty bin across the four iterations on tagged arithmetic (mean over 3 seeds). Endogenous variants fail by collapse ($r \approx 0$, iteration 1), inversion ($r = -0.59$, iteration 2), or instability (iteration 3); explicit supervision reaches $r = 0.95$.

Table 2: Tagged arithmetic: held-out accuracy (mean $\pm$ std over 3 seeds), balanced accuracy (macro over difficulty bins), compute per token, and gate spread (hardest minus easiest bin).

Method	Acc.	Balanced	MFLOPs/tok	Gate spread
Baseline	75.0 $\pm$ 0.9	83.8 $\pm$ 0.7	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Chemical-off	74.4 $\pm$ 0.7	83.4 $\pm$ 0.4	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Fixed-schedule (0.5)	72.3 $\pm$ 2.7	81.3 $\pm$ 2.3	19.0 $\pm$ 0.0	0.00 $\pm$ 0.00
MoD (top- k)	62.6 $\pm$ 4.8	72.3 $\pm$ 4.8	11.8 $\pm$ 0.0	-0.06 $\pm$ 0.01
Chemical (entropy)	71.5 $\pm$ 1.3	80.8 $\pm$ 1.3	11.5 $\pm$ 0.2	0.00 $\pm$ 0.00
Random-gate	70.7 $\pm$ 4.4	80.0 $\pm$ 4.2	11.2 $\pm$ 0.1	-0.00 $\pm$ 0.00
Tag-only	71.6 $\pm$ 3.1	80.9 $\pm$ 2.8	13.0 $\pm$ 0.4	-0.05 $\pm$ 0.09
Difficulty-supervised (ours)	74.6 $\pm$ 2.7	83.4 $\pm$ 2.1	19.3 $\pm$ 0.1	0.20 $\pm$ 0.03
Ours (Stage-1 = 3K)	66.9 $\pm$ 2.1	76.2 $\pm$ 2.4	19.6 $\pm$ 0.1	0.17 $\pm$ 0.03

Iteration 2: tag initialization inverts. Initializing the chemical state from the difficulty-tag embedding gives the pathway ground-truth information at the start of training. The gate does go live, but inverted: E/M/H gates are 0.266/0.265/0.213, and the per-problem correlation between mean gate and difficulty bin is -0.59 , negative in all 3 seeds. The cause is the training dynamics, not the initialization: the cross-entropy gradient is largest on easy (memorizable) tokens, so end-to-end training pushes compute where the loss is most reducible, not where the problem is hardest.

Iteration 3: a learned head without supervision is vacuous. Replacing the entropy pathway with the DifficultyHead of §3.1 uses a two-stage protocol (Stage 1 full FFN for 3000 steps; Stage 2 head trained only by cross-entropy), which yields a nearly flat gate: E/M/H = 0.812/0.824/0.834, spread 0.02. The per-problem gate-difficulty correlation is unstable across seeds ($r = 0.65/0.64/0.04$). Without an explicit signal, there is no reliable allocation.

Iteration 4: explicit supervision works. With the loss of Eq. (2), gates order correctly in every seed; we develop this result next. The contrast across iterations is the point: the failure is not in any one mechanism but in the assumption that endogenous training will discover the allocation semantics on its own.

4.3 Tagged arithmetic: main result

Table 2 reports the main tagged results. Our method reaches 74.6 $\pm$ 2.7% accuracy (83.4 $\pm$ 2.1 balanced) at 19.3 MFLOPs/token, 42% below the full-compute baseline’s 33.2. This statistically matches the baseline (75.0 $\pm$ 0.9) and the architecture-matched Chemical-off control (74.4 $\pm$ 0.7),

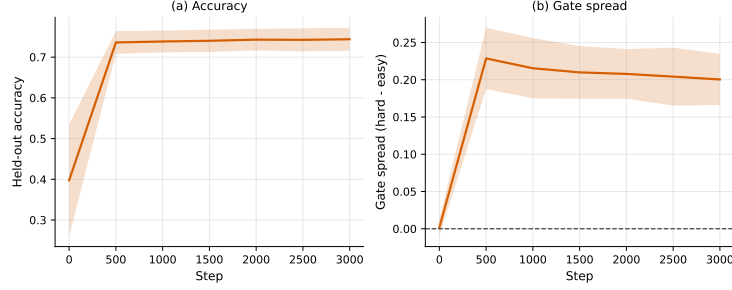


Figure 2: Stage-2 training on tagged arithmetic (mean over 3 seeds): held-out accuracy and gate spread (hardest minus easiest bin) versus step. The allocation emerges within the first few hundred Stage-2 steps and then stabilizes under the budget term; accuracy keeps climbing throughout.

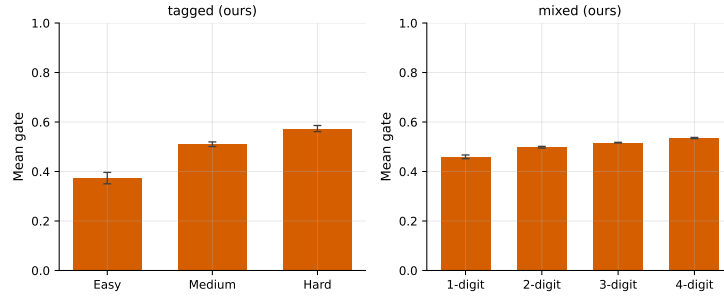


Figure 3: Mean gate per difficulty bin (E/M/H) on tagged arithmetic for our method and the failure-mode variants. Only explicit difficulty supervision produces the correct easy < medium < hard ordering consistently across seeds.

187 while the allocation is correct: mean gates for E/M/H are 0.37/0.51/0.58, ordered in all 3 seeds,
 188 with spread 0.20 ± 0.03 and per-problem gate-difficulty $r = 0.95$ (0.97/0.98/0.90 per seed; Fig-
 189 ure 3).

190 The allocation is learned quickly. Figure 2 shows the gate spread rising from zero to ~ 0.23 within
 191 the first few hundred Stage-2 steps and then settling near 0.20, while held-out accuracy continues
 192 to consolidate. A run stopped at only 500 Stage-2 steps already reaches per-problem $r = 0.988$
 193 (E/M/H gates 0.43/0.54/0.60); the remaining steps mostly buy back task accuracy under the budget
 194 constraint.

195 Two comparisons isolate the value of *adaptive* allocation. First, at the same compute budget,
 196 the Fixed-schedule baseline (gate 0.5 everywhere, 19.0 MFLOPs/token) reaches only $72.3 \pm 2.7\%$:
 197 spending the same FLOPs uniformly rather than by difficulty costs ~ 2 points. Second, static or unsu-
 198 pervised alternatives at even lower budgets, Random-gate (70.7 ± 4.4 @ 11.2M), Tag-only (71.6 ± 3.1
 199 @ 13.0M), and Chemical-entropy (71.5 ± 1.3 @ 11.5M), all trade accuracy away. Per-difficulty ac-
 200 curacies (Figure 4) show that our method preserves hard-bin accuracy while cutting compute on
 201 easy bins, which is where balanced accuracy is won.

202 4.4 Mixed arithmetic: partial generalization without tags

203 In the mixed regime there are no difficulty tags, so we supervise the head with a coarse proxy:
 204 digit count of the larger operand, mapping bins 1/2/3/4 to targets 0.2/0.4/0.6/0.8. (The mixed
 205 runs use a 3000-step Stage 1; only the tagged headline runs use 5000 steps.) The result is a par-
 206 tial success (Table 3, Appendix A). The allocation is correct in direction: mean gates per bin are
 207 0.45/0.50/0.52/0.53, monotone increasing, with per-problem $r = 0.70$, but the spread is small
 208 (0.08), and accuracy is $58.5 \pm 3.4\%$ (69.9 ± 2.7 balanced) at 19.6 MFLOPs/token, well below the
 209 6000-step baselines (85.0 ± 1.7 for the standard baseline).

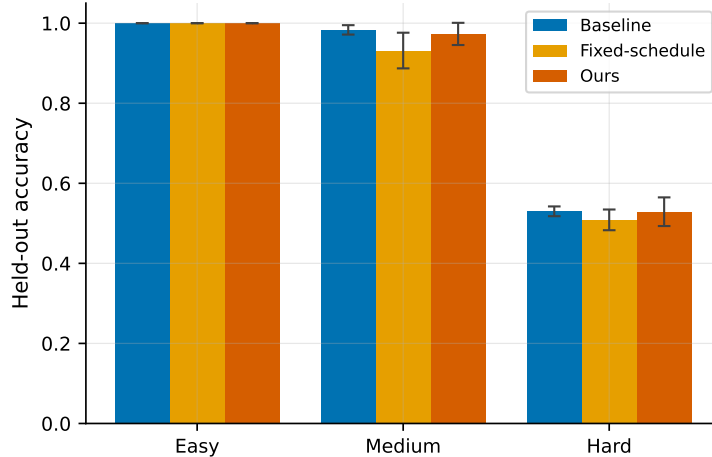


Figure 4: Per-difficulty held-out accuracy on tagged arithmetic. Our method keeps hard-bin accuracy close to the full-compute baseline while spending 42% fewer FLOPs per token; the fixed schedule at the same budget loses accuracy on the harder bins.

210 That gap needs context. Our mixed Stage-1-only checkpoint (3000 steps, full FFN, ungated) scores
 211 just 53.1% at 33.2 MFLOPs/token: the gated Stage-2 model *beats its own ungated starting point* by
 212 5.4 points while using 41% fewer FLOPs. Because the body is nearly frozen in Stage 2 (learning
 213 rate 3×10^{-6}), most of the gap to the baselines is a training-budget artifact rather than a cost of
 214 gating. We therefore claim generalization only in a limited sense: a coarse difficulty proxy suffices
 215 to produce a correctly ordered, positively correlated allocation on untagged inputs, but the allocation
 216 is shallow and the accuracy deficit is real. §5 returns to what would be needed to close it.

217 4.5 Comparisons and ablations

218 **Static schedules.** The Fixed-schedule baseline answers the question “why not just shrink the FFN
 219 uniformly?” At matched budget on tagged data it loses ~ 2 accuracy points to our method (Table 2);
 220 on mixed data it is strong (86.6 ± 0.9 @ 19.0M), indicating that when per-token difficulty information
 221 is weak or unavailable, a static schedule is a surprisingly competitive default (see §5).

222 **MoD inverts too.** The inversion pathology is not specific to our architecture. A standard MoD top- k
 223 router [Raposo et al., 2024] allocates a slightly *larger* fraction of its compute budget to easy tokens
 224 than to hard ones (fraction of tokens processed, E/M/H = 0.135/0.109/0.093), and its accuracy
 225 suffers (62.6 ± 4.8 tagged, 70.5 ± 2.9 mixed). Endogenous training dynamics push compute toward
 226 reducible error regardless of the routing mechanism. MoD is thus not a counterexample to our
 227 diagnosis but its most prominent instance: the canonical token-level allocator rests on the same
 228 untested assumption, and in our setting it fails the same way.

229 **Controls.** Random-gate (70.7 ± 4.4) and Tag-only (71.6 ± 3.1) show that neither gate variance alone
 230 nor a static oracle tag heuristic explains our result; Chemical-off (74.4 ± 0.7) confirms the gating
 231 pathway itself costs nothing when disabled.

232 **Stage-1 length.** Ablating Stage 1 from 5000 to 3000 steps (same protocol otherwise) drops tagged
 233 accuracy to 66.9 ± 2.1 @ 19.6 MFLOPs/token (Table 2, “Ours (Stage-1 = 3K)”: the body needs
 234 enough Stage-1 training before the allocation layer is added, consistent with the mixed-regime anal-
 235 ysis of §4.4.

236 **Frontier.** Figure 5 plots accuracy against MFLOPs per token for all methods on both regimes. On
 237 tagged data our method sits on the Pareto frontier at the high-accuracy end, the only adaptive method
 238 that matches full-compute accuracy at substantially reduced compute.

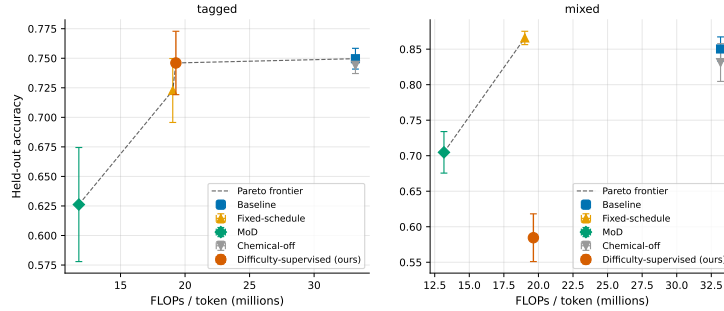


Figure 5: Accuracy–compute frontier on tagged and mixed arithmetic (3 seeds). On tagged data, difficulty-supervised gating matches full-compute accuracy at 42% fewer FLOPs per token and dominates the same-budget static schedule; on mixed data it improves over its own Stage-1 starting point but trails the longer-trained baselines.

5 Discussion

Implications for resource-aware agents. Our experiments use arithmetic as a controlled testbed, but the core finding, that endogenous difficulty signals fail and explicit supervision is required, should carry to any agentic reasoning loop whose tokens differ in computational demand, from tool-use trajectories to code generation. The practical lesson is about where difficulty knowledge comes from. Agentic pipelines often possess difficulty metadata that end-to-end training cannot discover on its own: curriculum tags in training corpora, verifier or reward-model scores, problem metadata, or cheap proxy features such as operand length. Our results show that spending this metadata on explicit per-token supervision yields an allocation that is reliable (correct ordering in every seed), interpretable (gates can be read directly as a compute budget per token), and cheap (a two-layer head per transformer layer). Conversely, relying on signals the network already produces, entropy and router scores, risks allocation that is absent, inverted, or unstable, and the failure can be silent: iteration 1 saved 65% of FLOPs with healthy accuracy while allocating nothing differentially. Inversion is the more dangerous failure than collapse: it actively misallocates while appearing to work.

When static schedules suffice. On mixed data the fixed schedule at the same budget beat every adaptive method (Table 3). Adaptive allocation pays when (i) difficulty varies substantially across tokens, (ii) a reliable difficulty signal or proxy exists, and (iii) the base model is already competent. When any of these fails, a weak proxy in the mixed regime or an undertrained body from a short Stage 1, a uniform schedule is a strong baseline that adaptive methods must beat, not merely undercut on FLOPs.

Limitations. Our study is deliberately narrow: a single task family (arithmetic), a single 11M-parameter model, and difficulty defined by operand size; whether the same failure modes and fixes hold at scale and on natural language is open. FLOPs are an accounting of skippable compute: under dense execution the wallclock is unchanged, and realizing the savings requires sparse or gated kernels. The mixed-regime gap is partly a training-budget artifact, and our tagged comparisons are same-task, similar-budget rather than iso-FLOPs controlled. Our targets are hand-set (0.2/0.5/0.8); learning them, or deriving them from measured per-problem loss, is a natural next step.

6 Conclusion

We tested the assumption underlying most token-level adaptive-compute methods: that endogenous signals correlate with problem difficulty. On arithmetic with graded difficulty, they do not: they fail by collapse, inversion, or vacuity across three distinct mechanisms, with a standard MoD router inverting as well. Explicit per-token difficulty supervision with a budget constraint fixes the allocation: correct ordering in every seed, per-problem $r = 0.95$, full-compute accuracy at 42% fewer FLOPs per token, and a win over the same-budget static schedule. For resource-aware agents the message is conservative but useful: when difficulty labels or proxies exist, supervised token-level allocation

is a simple, reliable, interpretable metering primitive; when they do not, endogenous signals should not be trusted to allocate on their own.

References

- Lingjiao Chen, Matei Zaharia, and James Zou. Frugalpt: How to use large language models while reducing cost and improving performance, 2023.
- Kevin Clark, Urvashi Khandelwal, Omer Levy, and Christopher D. Manning. What does bert look at? an analysis of bert’s attention, 2019.
- Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer, 2020.
- Alex Graves. Adaptive computation time for recurrent neural networks, 2016.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks, 2017.
- David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models, 2024.
- Tal Schuster, Adam Fisch, Jaakkola Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling, 2022.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, 2017.
- Surat Teerapittayanon, Bradley McDanel, and H. T. Kung. Branchynet: Fast inference via early exiting from deep neural networks, 2017.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2017.
- Jesse Vig and Yonatan Belinkov. Analyzing the structure of attention in a transformer language model, 2019.
- Xin Wang, Fisher Yu, Zi-Yi Dou, Trevor Darrell, and Joseph E. Gonzalez. Skipnet: Learning dynamic routing in convolutional networks, 2018.
- Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. Deebert: Dynamic early exiting for accelerating bert inference, 2020.
- Jiahui Yu and Thomas S. Huang. Universally slimmable networks and improved training techniques, 2019.

A Full mixed-arithmetic results

Table 3 reports per-method results for the mixed regime; §4.4 summarizes and interprets them in the main text.

B Experimental details

Model and optimization. All models share the 11M-parameter decoder-only backbone of §3.1 ($d_{\text{model}} = 384$, 6 layers, 6 heads, $d_{\text{ff}} = 1536$, vocab 22, sequence length 256). We use batch size 32, AdamW, and gradient clipping at 1.0 throughout. Stage 2 uses two parameter groups: body at 3×10^{-6} , DifficultyHeads at 3×10^{-4} . The loss is Eq. (2) with $\lambda_{\text{mse}} = 5.0$, $\lambda_{\text{budget}} = 0.5$, budget target 0.5, and no sparsity penalty. Tagged targets are $E \rightarrow 0.2$, $M \rightarrow 0.5$, $H \rightarrow 0.8$ with non-problem positions at 0.5; mixed targets map digit bins 1/2/3/4 to 0.2/0.4/0.6/0.8.

Table 3: Mixed arithmetic: held-out accuracy (mean $\pm$ std over 3 seeds), balanced accuracy (macro over difficulty bins), compute per token, and gate spread (hardest minus easiest bin).

Method	Acc.	Balanced	MFLOPs/tok	Gate spread
Baseline	85.0 $\pm$ 1.7	90.6 $\pm$ 1.0	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Chemical-off	83.1 $\pm$ 2.7	89.0 $\pm$ 2.0	33.2 $\pm$ 0.0	0.00 $\pm$ 0.00
Fixed-schedule (0.5)	86.6 $\pm$ 0.9	91.3 $\pm$ 0.7	19.0 $\pm$ 0.0	0.00 $\pm$ 0.00
MoD (top- k)	70.5 $\pm$ 2.9	78.5 $\pm$ 2.9	13.2 $\pm$ 0.0	-0.03 $\pm$ 0.05
Chemical (entropy)	84.1 $\pm$ 1.2	89.8 $\pm$ 0.8	12.8 $\pm$ 0.4	-0.00 $\pm$ 0.00
Random-gate	82.8 $\pm$ 4.4	88.8 $\pm$ 3.0	12.2 $\pm$ 0.8	0.00 $\pm$ 0.00
Difficulty-supervised (ours)	58.5 $\pm$ 3.4	69.9 $\pm$ 2.7	19.6 $\pm$ 0.1	0.08 $\pm$ 0.01

Evaluation protocol. The held-out set contains 4000 problems per difficulty bin, generated once with seed 12345 and shared across all methods and seeds. Accuracy is computed on answer tokens only; balanced accuracy is the macro average over bins. Gate statistics are means over the layer-averaged gate at problem positions. Reported correlations are per-problem Pearson r between the mean gate of a problem’s positions and its difficulty bin, except where noted as per-token (iteration 1’s entropy analysis). All numbers are mean $\pm$ std over 3 seeds.

Per-difficulty details. Per-bin accuracies for all methods are visualized in Figure 4; per-bin gates in Table 1 and Figure 3. Gate spread is defined as the hardest-bin mean gate minus the easiest-bin mean gate.

Code. An anonymized implementation is available at <https://anonymous.4open.science/r/chemical-transformer-BB35/>.